

1. Basic file structure (e.g. workspaces) of ROS

The new build system for ROS is "catkin", while "roscpp" is the old ROS build system which was replaced by catkin.

Packages in a catkin Workspace:

```
workspace_folder/      -- WORKSPACE
  src/                  -- SOURCE SPACE
    CMakeLists.txt      -- 'Toplevel' CMake file, provided by catkin
  package_1/
    CMakeLists.txt      -- CMakeLists.txt file for package_1
    package.xml         -- Package manifest for package_1
  ...
  package_n/
    CMakeLists.txt      -- CMakeLists.txt file for package_n
    package.xml         -- Package manifest for package_n
```

Create a workspace:

Code:

```
source /opt/ros/noetic/setup.bash
mkdir -p ~/catkin_ws/src
cd ~/catkin_ws/

catkin_make (Running it the first time in your workspace, it will create
a CMakeLists.txt link in your 'src' folder.)

source devel/setup.bash (Before continuing, source your new setup.*sh
file.)
```

Filesystem Tools:

i) Rospack: Rospack allows you to get information about packages.

Code: rospack find [package_name]

Would return: YOUR_INSTALL_PATH/share/roscpp

- ii) Roscd: Roscd allows you to change directory (cd) directly to a package or a stack without mentioning the path.

Code: roscd <package-or-stack>

- iii) Rosls: Rosls allows you to ls directly by name rather than its absolute path.

Code: rosls <package-or-stack>

2. Operating procedure of ROS (Mention the key points, e.g. building the workspace, running roscore etc.)

- i) Creating a catkin Package

cd ~/catkin_ws/src

catkin_create_pkg beginner_tutorials std_msgs rospy roscpp (Create a new package called 'beginner_tutorials' which depends on std_msgs, roscpp, and rospy. This will also create a beginner_tutorials folder.)

- ii) Building a catkin workspace

cd ~/catkin_ws

catkin_make

. ~/catkin_ws/devel/setup.bash (To add the workspace to ROS environment)

- iii) Starting ROS

roscore (Master + rosout + parameter server. Master is name service for ROS which helps nodes find each other. A node is an executable which uses ROS to communicate with other nodes. Rosout is ROS equivalent of stdout/stderr.)

3. Explain ROS nodes and how they are used to transmit data around ROS

A node really isn't much more than an executable file within a ROS package. Nodes can publish or subscribe to a Topic. Nodes can also provide or use a Service.

Rosnodes can be written in different programming languages –

- Rospy = python client library
- Roscpp = c++ client library

Using rosnode:

i) rosnode displays information about the ROS nodes that are currently running. The 'rosgrep' command lists these active nodes.

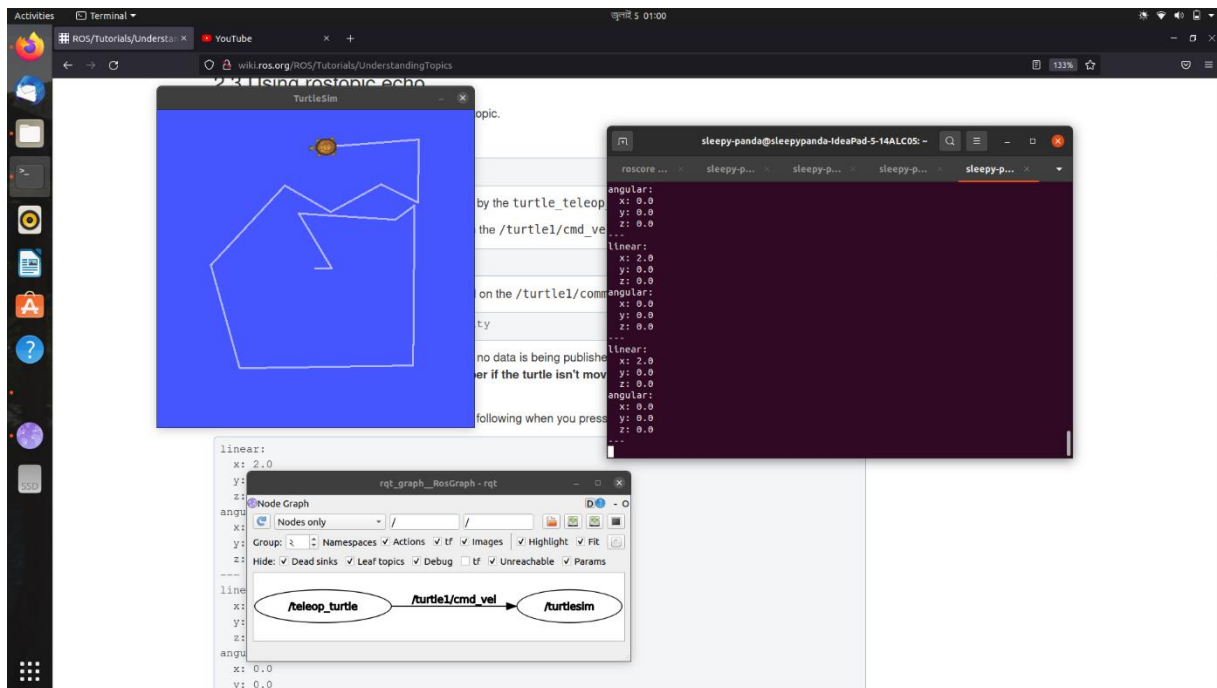
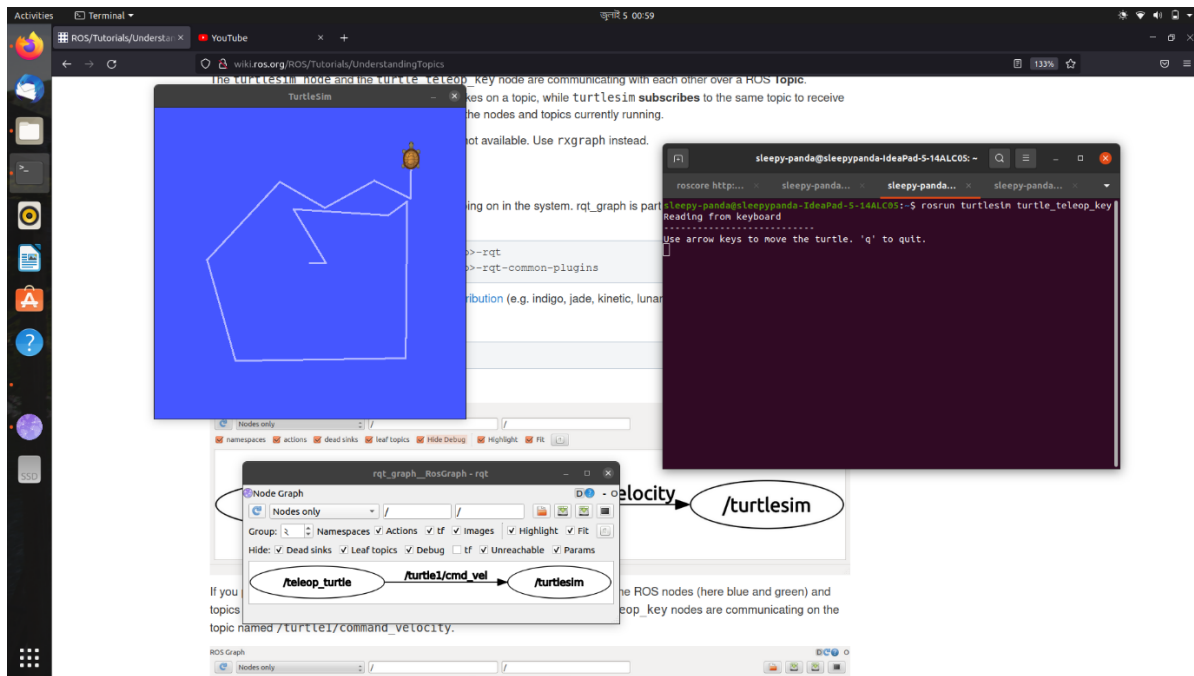
ii) The 'rostopic info' command returns information about a specific node.

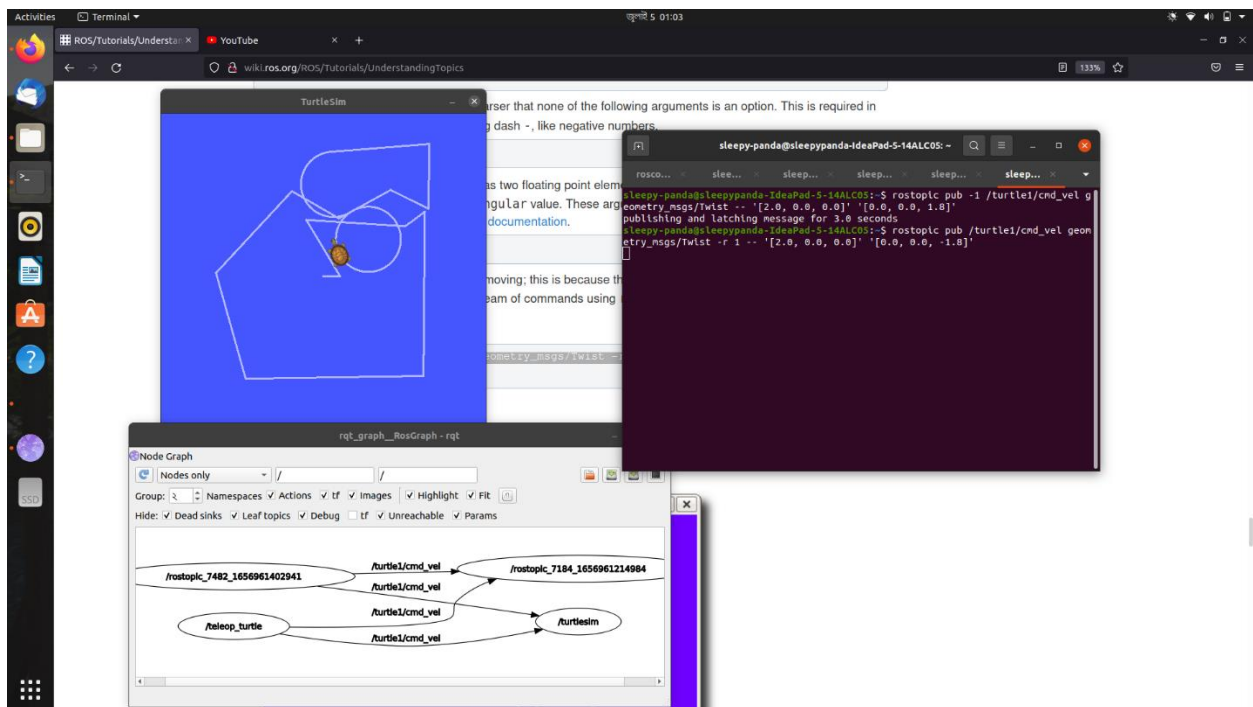
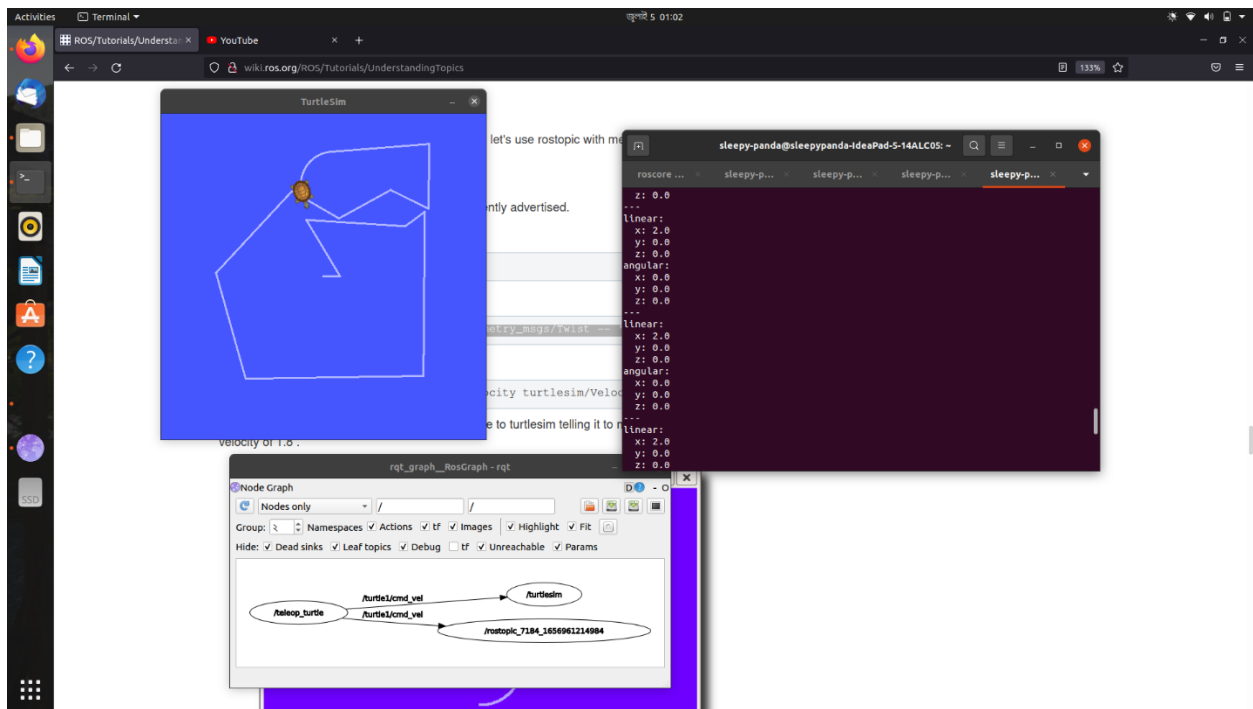
iii) roslaunch allows you to use the package name to directly run a node within a package (without having to know the package path).

Code: `roslaunch [package_name] [node_name]`

4. Some basic simulations in the online simulator

Using turtlebot simulation:





5. Shortcomings about ROS

- i) I think a nominal GUI would have been nice for this operating system.
- ii) Not having 100% dependency in Windows OS.
- iii) I think it is pretty backdated to have unique dependencies in Arduino sketches (std:msgs and all those variable initialization things. It could have been made simpler by using C dependencies.)